EXPERIMENT 9

Developing a Simple Containerized Application Using Docker

Python Flask Application on AWS EC2 Ubuntu using Docker Swarm

Name: Aayush Sujaykuamr Patil Roll No: A31

Division: B.TECH A PRN : 2324000488

1. Aim

To develop a simple Python Flask web application, package it into a Docker image, and deploy it as a Docker Swarm service on an AWS EC2 Ubuntu instance.

2. Objectives

- Verify Docker Engine on the EC2 instance.
- Create a simple Flask web application.
- Define the application environment using a Dockerfile.
- Build and verify a Docker image.
- Initialize Docker Swarm and deploy the image as a service.
- Verify service status and test the application response.

3. Software and Requirements

AWS EC2 virtual machine running Ubuntu 26.04.1 LTS; Docker Engine and CLI; internet access to retrieve the base image and install Flask; SSH or EC2 Instance Connect terminal; inbound TCP port 5000 access if the application must be accessed externally.

4. Theory

Docker packages an application and its dependencies into an image. A container is an instance created from an image. A Dockerfile contains instructions for building the image. Docker Swarm is Docker's orchestration mode; a Swarm service defines the desired state of an application and manages its task replicas.

In this experiment, a Flask application responds on port 5000. The image is built on the EC2 instance and deployed as a replicated Swarm service. The service publishes host port 5000 to container port 5000.

5. Procedure

Step 1: Verify Docker and the EC2 environment

```
sudo docker --version
sudo docker info
```

The terminal output showed Docker Engine version 29.9.2. The instance reported Ubuntu 26.04.1 LTS, x86_64 architecture, 2 CPUs and approximately 908 MiB of memory.

Step 2: Initialize Docker Swarm

```
sudo docker swarm init
sudo docker node ls
```

Swarm initialization completed successfully. The node was listed as Ready and Active, with Manager Status shown as Leader. The Swarm join token is sensitive and is not included in this report.

Step 3: Create the project directory

```
mkdir -p ~/docker-experiment
cd ~/docker-experiment
```

The project directory was created to store the application source code and Dockerfile.

Step 4: Create the Flask application

```
cat > app.py <<'EOF'
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello! My first Docker container is running."

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
EOF
```

The root route (/) returns a plain-text message. Binding to 0.0.0.0 allows the server to accept connections through the container network interface, and port 5000 is used by Flask.

Step 5: Create the Dockerfile

```
cat > Dockerfile <<'EOF'
FROM python:3.12-slim

WORKDIR /app

RUN pip install --no-cache-dir flask

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
EOF
```

FROM selects a lightweight Python base image; WORKDIR sets the working directory; RUN installs Flask; COPY adds app.py; EXPOSE documents the application port; CMD starts Flask.

Step 6: Build and verify the Docker image

```
sudo docker build -t my-python-app .
sudo docker images
```

The image build completed successfully and the image my-python-app:latest appeared in the image list.

Step 7: Create the Docker Swarm service

```
sudo docker service create \
  --name my-python-service \
  --publish published=5000,target=5000 \
  my-python-app
```

Docker created the service and reported that it converged. Since the image was not pushed to a registry, Docker warned that other Swarm nodes would need to access the image independently. This is acceptable for the single-node demonstration; a multi-node deployment should use a registry accessible to all nodes.

Step 8: Verify the service

```
sudo docker service ls
```

The service list showed my-python-service in replicated mode with 1/1 replicas and port mapping *:5000->5000/tcp. The command `docker services ls` is invalid; the correct command is `docker service ls` (singular).

Step 9: Test the application

```
curl http://localhost:5000
```

Observed response:

```
Hello! My first Docker container is running.
```

This confirms that the deployed service responded successfully from the EC2 instance.

6. Result

The Python Flask application was successfully containerized using Docker. The Docker image my-python-app:latest was built, Docker Swarm was initialized, and the application was deployed as my-python-service with one running replica. The endpoint returned the expected message when tested using curl on localhost port 5000.

7. Screenshots and Evidence

The screenshots are arranged to support the procedure. Captions describe the visible evidence. The supplied screenshots do not show a browser opened from an external computer, so the report only claims the verified localhost curl test.

Figure 1. Docker Swarm service verification and application response



Figure 1: Docker Swarm service verification and application response.

The service listing shows my-python-service in replicated mode with 1/1 replicas and port mapping 5000:5000. The curl request to localhost:5000 returns the expected Flask message.

Figure 2. Service creation and successful convergence

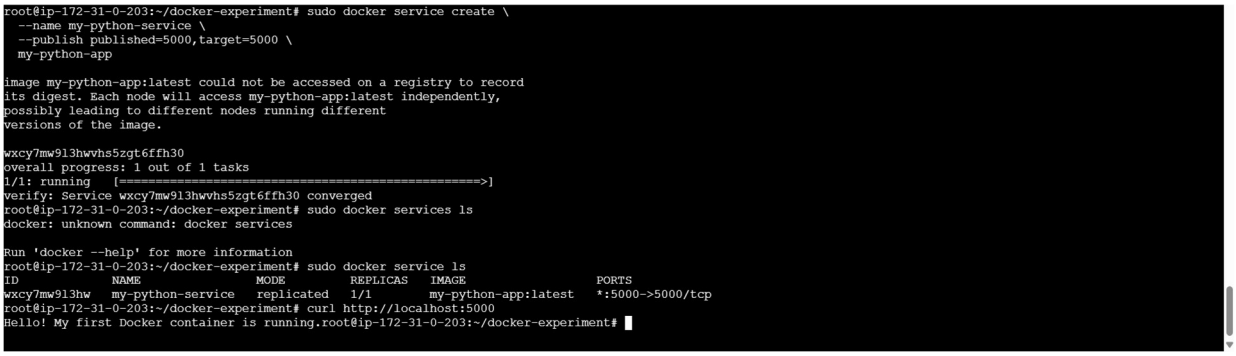


Figure 2: Service creation and successful convergence.

Docker creates the my-python-service Swarm service. The output reports that the single task is running and the service has converged. The image-registry warning is expected for this single-node demonstration.

Figure 3. Service listing and local endpoint test

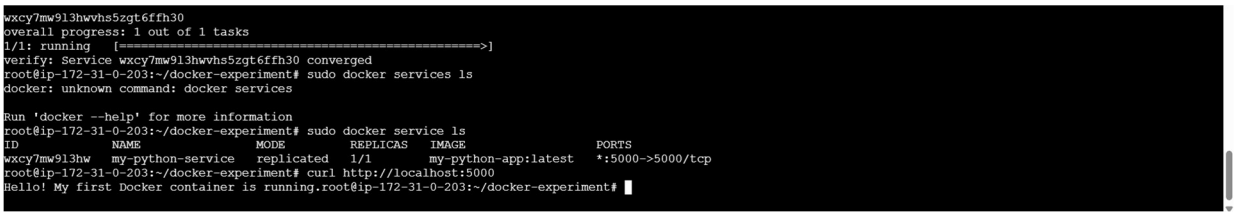


Figure 3: Service listing and local endpoint test.

The correct command docker service ls displays the service and its 1/1 replica state. The curl command confirms that the application responds on port 5000.

Figure 4. Docker image listing



Figure 4: Docker image listing.

The local Docker image list contains my-python-app:latest, confirming that the image is available after the build.

Figure 5. Successful Docker image build



Figure 5: Successful Docker image build.

The Docker build completes successfully, using python:3.12-slim as the base image, installing Flask, copying app.py, and exporting the my-python-app image.

Figure 6. Docker Swarm initialization and node status

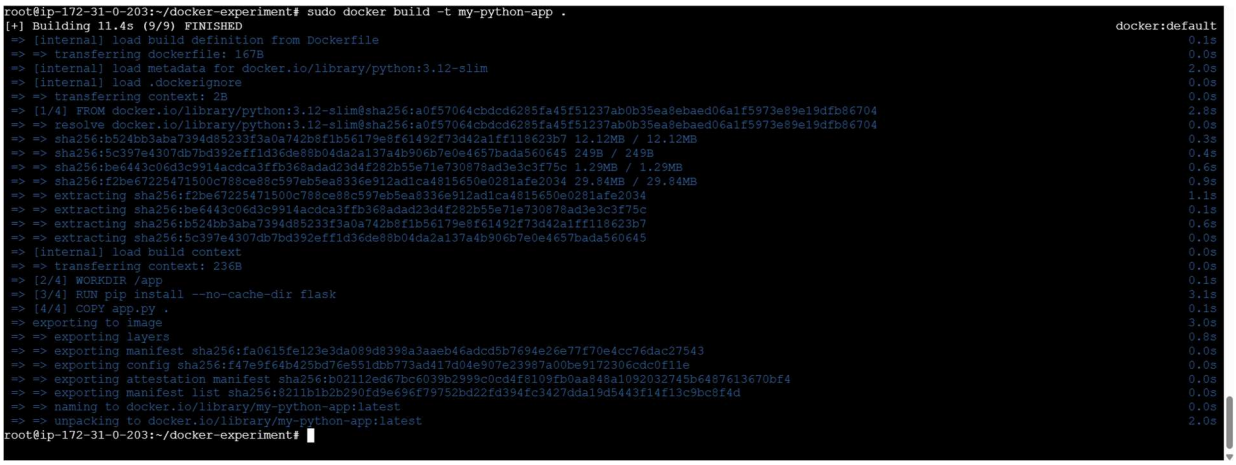


Figure 6: Docker Swarm initialization and node status.

Docker Swarm initializes successfully. The node list shows the EC2 instance as Ready and Active, with Manager Status Leader. The join token is not reproduced in this report.

Figure 7. Dockerfile contents

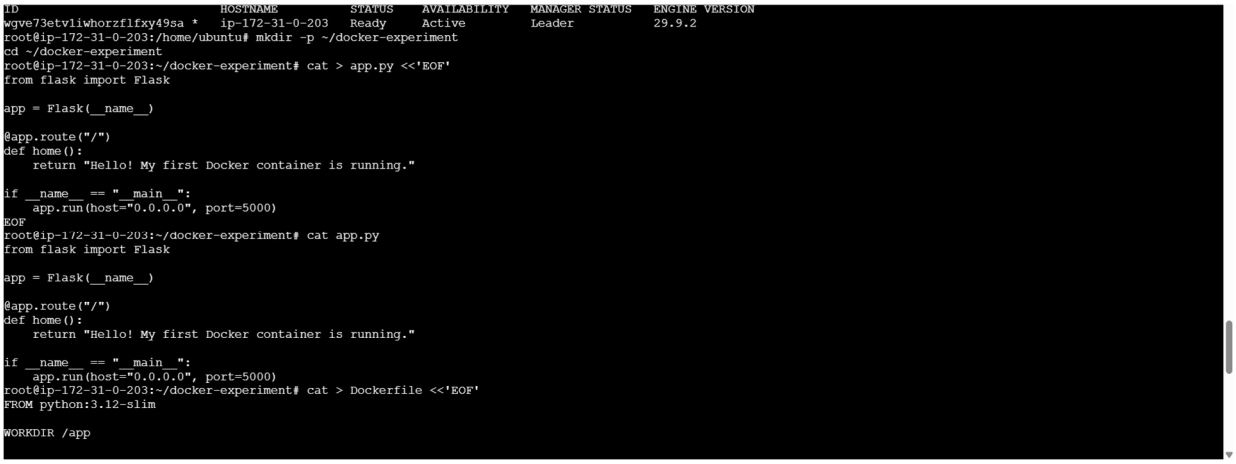


Figure 7: Dockerfile contents.

The Dockerfile uses python:3.12-slim, sets /app as the working directory, installs Flask, copies app.py, documents port 5000, and starts the application.

Figure 8. Flask application source code

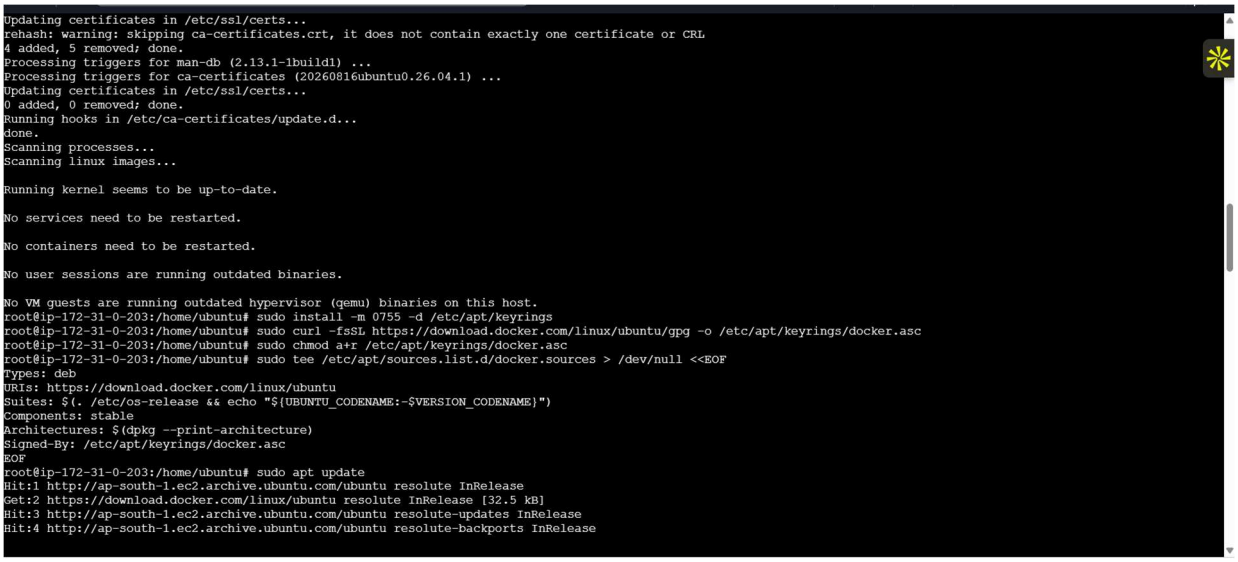


Figure 8: Flask application source code.

The app.py file defines a Flask application with a root route that returns the message 'Hello! My first Docker container is running.' and listens on 0.0.0.0:5000.

Figure 9. Docker Engine installation and package setup

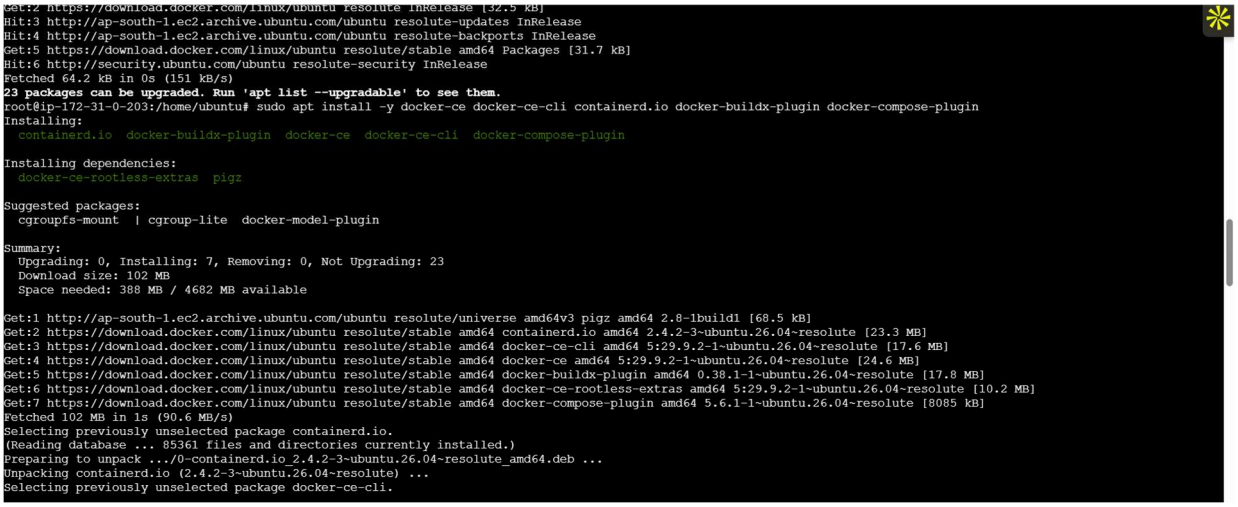


Figure 9: Docker Engine installation and package setup.

The terminal shows the Docker repository being configured and Docker Engine packages being installed on the Ubuntu EC2 instance.

Figure 10. Docker installation package progress



Figure 10: Docker installation package progress.

The package manager downloads Docker Engine, the CLI, containerd, Buildx, and the Docker Compose plugin.

Figure 11. Docker system information

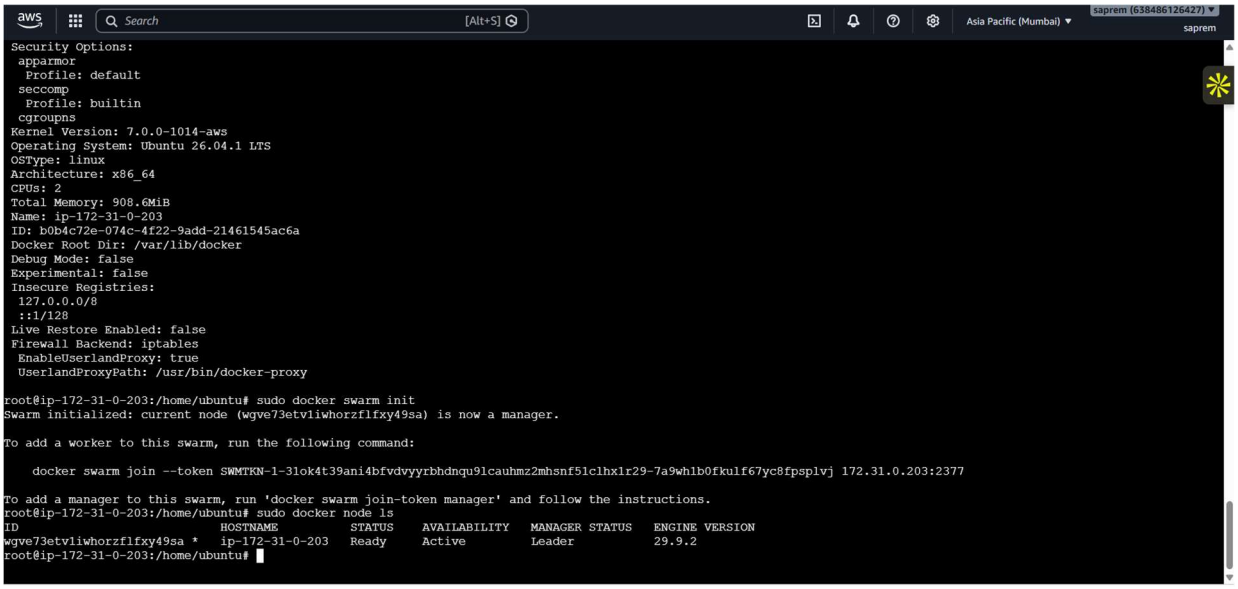


Figure 11: Docker system information.

Docker system information confirms the EC2 environment is Ubuntu Linux on x86_64 with 2 CPUs and approximately 908 MiB of memory.

8. Conclusion

The experiment demonstrated the basic containerization workflow using Docker on AWS EC2. A Python Flask application was written, packaged into a Docker image, and deployed as a Docker Swarm service. The service status showed one running replica, and the expected application message was returned through the local endpoint.